SMART FIRE DETECTION & REMOTE MONITORING SYSTEM

Project Delivery Report

ESP32 embedded alarm • Raspberry Pi monitoring server • Secure Internet transport

EDGE DEVICE	SERVER	CONNECTIVITY
ESP32	**Raspberry Pi**	**Cloudflare Tunnel**
Local detection and alarm logic	Node.js / Express dashboard	HTTPS between different networks

Status: Functional prototype implemented and tested

Author: Iván Baltanás Ortiz

Index

1. Executive Summary 3
2. Project Objectives 3
3. Final Scope and Implementation Status 3
4. Hardware Architecture 4
4.1 Hardware Components 4
4.2 Local Operating States 5
5. ESP32 Firmware Design 5
5.1 Firmware Module Structure 5
5.2 Main Control Flow 6
5.3 Telemetry Timing 6
6. Raspberry Pi Server and Web Dashboard 6
6.1 Server Endpoints 6
6.2 OFF-State Data Retention 6
6.3 Dashboard Behaviour 6
7. Network Architecture and Security 7
7.1 Security Controls Implemented 7
8. Testing and Validation 8
9. Key Design Decisions 9
10. Limitations and Remaining Work 9
11. Conclusion 10
Appendix A. Data Model 10
Appendix B. Technology Stack 10

1. Executive Summary

This project implements a smart fire-detection prototype that combines local embedded detection with remote real-time monitoring. An ESP32 reads environmental and fire-related sensors, determines the current alarm state locally, and controls visual and acoustic outputs without depending on an Internet connection. A Raspberry Pi receives the current status over the Internet and serves a private web dashboard for real-time monitoring.

The design intentionally avoids persistent storage. The Raspberry Pi keeps only the latest received data in memory. When the alarm is switched OFF, the ESP32 stops evaluating the sensors and the server preserves the last valid sensor readings while showing that the system is OFF. Communication between the ESP32 and Raspberry Pi is secured with HTTPS, certificate validation, a shared API key, strict request validation, and a Cloudflare Tunnel so that the devices can operate from different networks without opening inbound router ports.

2. Project Objectives

- Detect abnormal environmental conditions using temperature, smoke/gas and flame sensing.
- Provide immediate local feedback using an LCD, RGB status lighting and an audible buzzer.
- Support clearly defined operating states: NORMAL, WARNING, CRITICAL and OFF.
- Allow the user to disable or enable sensing with a physical switch.
- Transmit the current alarm status securely from the ESP32 to a Raspberry Pi over the Internet.
- Display the live system state and latest sensor readings through a web dashboard accessible only from the Raspberry Pi local network or VPN.
- Keep the embedded alarm autonomous: loss of network connectivity must not disable local detection or alarm outputs.

3. Final Scope and Implementation Status

Feature	Status	Notes
Temperature and humidity sensing	Implemented	DHT11 connected to ESP32.
Smoke/gas sensing	Implemented	Analog MQ sensor input; calibration remains a final tuning task.
Flame sensing	Implemented	KY-026 analog flame sensor.
Local LCD display	Implemented	Shows temperature, humidity and system state.
RGB status indication	Implemented	Green / yellow / red / off according to state.
Audible critical alarm	Implemented	Active buzzer driven through S8050 transistor.
Physical ON/OFF control	Implemented	Slide switch; OFF mode ignores sensor evaluation.
Internet telemetry	Implemented	ESP32 sends state/data to Raspberry Pi through HTTPS.

Feature	Status	Notes
Cloudflare Tunnel	Implemented	Raspberry and ESP32 can be on different networks.
Private real-time dashboard	Implemented	Dashboard served locally/VPN from Raspberry Pi.
Persistent database	Not in scope	Latest values are stored only in RAM by design.
Final sensor calibration	Pending final tuning	Thresholds are currently prototype/test values.

4. Hardware Architecture

The ESP32 is the central controller. Sensors provide input signals, while the LCD, RGB module and buzzer provide local feedback. The physical switch controls whether the alarm is actively evaluating the sensors.

4.1 Hardware Components

Component	Purpose	ESP32 connection	Important design note
ESP32	Main embedded controller	—	Runs the alarm logic locally.
DHT11	Temperature and humidity	GPIO 13	Powered from 3.3 V.
MQ smoke/gas module	Smoke/gas level	GPIO 34 (ADC1)	Analog signal conditioned before ESP32 ADC input; sensor requires warm-up/calibration.
KY-026 flame sensor	Flame / infrared response	GPIO 35 (ADC1)	Analog value decreases as detected flame intensity increases in the tested setup.
16x2 I²C LCD	Local data/state display	SDA GPIO 21, SCL GPIO 22	Powered from 5 V; I²C reduces required GPIO count.
Freenove 8 RGB LED module	Visual state indication	GPIO 2	Addressable WS2812 LEDs; one signal line controls all eight.
Active buzzer	Critical audible alarm	GPIO 18 through S8050	GPIO drives transistor base through 1 kΩ resistor; buzzer current does not flow directly through GPIO.
S8050 NPN transistor	Buzzer switching	Base controlled by GPIO 18	Emitter to GND; collector in buzzer current path.
Slide switch	System active/OFF selection	GPIO 19	Configured with INPUT_PULLUP; one switch position pulls the input to GND.
Raspberry Pi	Remote monitoring server	Network connection	Runs Node.js/Express and the private web dashboard.

4.2 Local Operating States

State	Meaning	RGB	Buzzer
NORMAL	No configured threshold exceeded	Green	Off
WARNING	At least one warning condition detected	Yellow	Off
CRITICAL	At least one critical condition detected	Red	On
OFF	Sensor evaluation disabled by switch	Off	Off

5. ESP32 Firmware Design

The firmware is written in C++ using the Arduino framework. During development it was refactored from one large sketch into small modules so that hardware control, state logic, debugging and networking are separated.

5.1 Firmware Module Structure

Module	Responsibility
alarmSmartTech.ino	Main orchestration: setup, switch handling, sensor-read cycle, state update and module calls.
config.h	Pin assignments, state enumeration, timing constants and prototype sensor thresholds.
sensors.h / sensors.cpp	SensorData structure, sensor initialization and acquisition.
system_state.h / system_state.cpp	Determines NORMAL, WARNING or CRITICAL from sensor data.
outputs.h / outputs.cpp	LCD, RGB LED module and buzzer initialization/output control.
debug.h / debug.cpp	Serial Monitor diagnostic output.
network.h / network.cpp	Wi-Fi connection, JSON construction, HTTPS POST requests and heartbeat transmission.
secrets.h	Wi-Fi credentials, API URL and API key; excluded from Git.

certificates.h	Trusted root CA certificate used for TLS server validation.

5.2 Main Control Flow

The main loop first checks the physical switch. If the system is OFF, local alarm outputs are disabled, sensor evaluation is skipped, and an OFF status is periodically sent to the Raspberry Pi. If the system is active, sensor data is read, the state is calculated locally, outputs are updated, diagnostics are printed and telemetry is sent when needed.

```
switch state -> OFF?
  yes: state=OFF -> RGB off -> buzzer off -> LCD OFF -> send OFF heartbeat -> return
  no:  read sensors -> determine state -> update LCD/RGB/buzzer -> send telemetry
```

5.3 Telemetry Timing

The ESP32 does not send a request for every small sensor fluctuation. Telemetry is sent immediately when the alarm state changes and otherwise at a periodic interval (currently 10 seconds). This keeps the dashboard current while avoiding unnecessary network traffic.

6. Raspberry Pi Server and Web Dashboard

The Raspberry Pi runs a lightweight Node.js server using Express. No database is used. The server stores only the latest known alarm values in Raspberry's RAM and returns them to the dashboard through a GET endpoint.

6.1 Server Endpoints

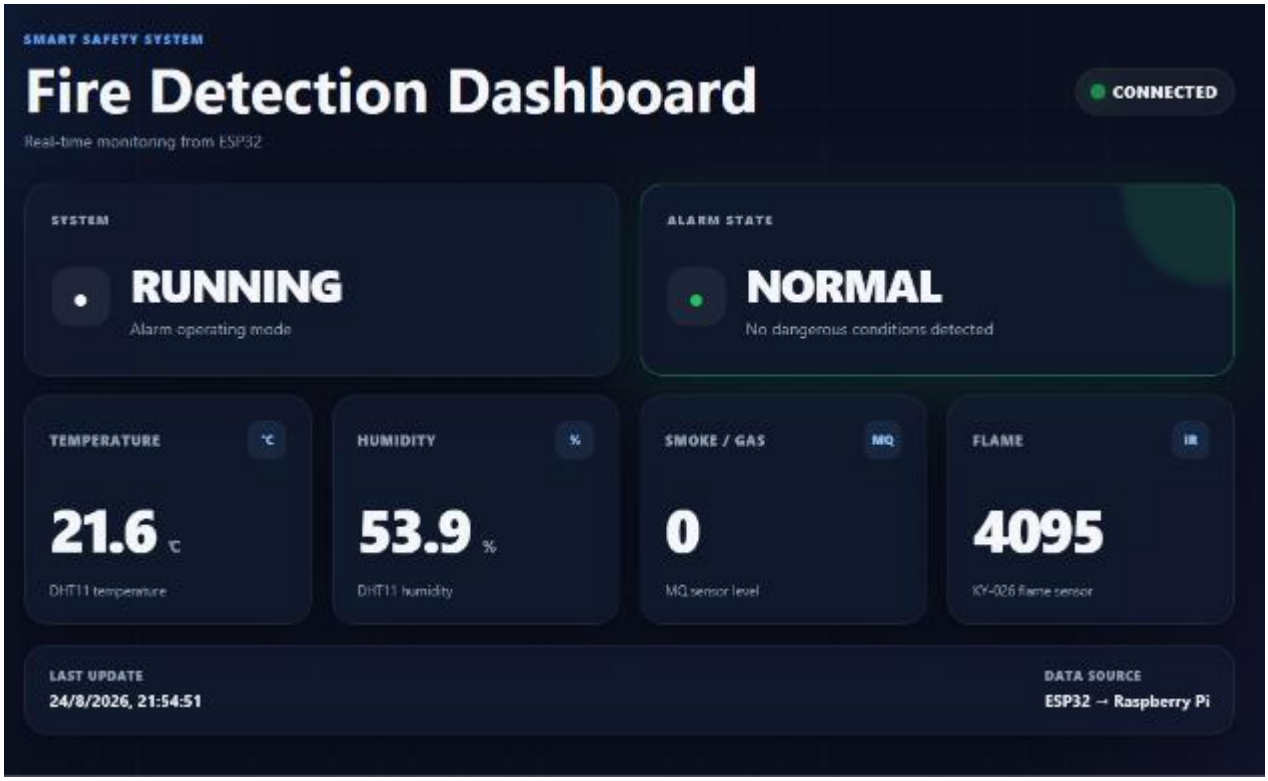
Method	Path	Exposure	Purpose
POST	/api/status	Internet through Cloudflare Tunnel	Receives authenticated telemetry from the ESP32.
GET	/api/status	Local/VPN dashboard usage	Returns the latest in-memory values.
GET	/	Local/VPN only	Serves the monitoring dashboard from the public folder.

6.2 OFF-State Data Retention

When RUNNING telemetry is received, the server replaces all current readings. When an OFF message is received, only system, state and last-update information are changed. Temperature, humidity, smoke and flame remain at their last valid values. This allows the dashboard to show the last readings that existed before sensing was disabled.

6.3 Dashboard Behaviour

The dashboard is built with HTML, CSS and browser-side JavaScript. It polls GET /api/status once per second. The interface displays system mode, alarm state, sensor values, last update time and connection status. Modern visual styles and state-change animations provide clear feedback when the alarm moves between NORMAL, WARNING, CRITICAL and OFF.



7. Network Architecture and Security

The ESP32 and Raspberry Pi operate on different networks. Cloudflare Tunnel provides an outbound-only connection from the Raspberry Pi to Cloudflare, avoiding router port forwarding. The public hostname is used only for the telemetry API, while the dashboard remains accessible only through the Raspberry Pi local network or existing VPN.

ESP32 → HTTPS → Cloudflare Edge → Cloudflare Tunnel → Raspberry Pi / Express

7.1 Security Controls Implemented

Control	Purpose
HTTPS encryption	Sensor data and the API key are encrypted in transit.
TLS certificate validation	ESP32 validates the server certificate chain using a trusted CA rather than setInsecure().
API key authentication	POST requests must include the expected X-API-Key header.
Secret separation	Raspberry secrets are stored in .env; ESP32 credentials are stored in secrets.h; both are excluded from Git.
Input validation	The API validates JSON presence, data types,

	permitted state/system strings and expected sensor ranges.
Generic error responses	Unexpected server errors return a generic JSON message instead of exposing stack traces or local filesystem paths.
Express fingerprint reduction	The X-Powered-By header is disabled.
No inbound router port	Cloudflare Tunnel avoids direct Internet exposure of the Raspberry Pi service port.
Private dashboard	Only the telemetry API is intended for public reachability; the monitoring UI stays local/VPN.

8. Testing and Validation

The system was developed incrementally. Each hardware component was tested independently before being integrated, reducing troubleshooting complexity.

Test	Observed result
DHT11	Temperature and humidity values displayed correctly in Serial Monitor and LCD.
RGB module	All eight WS2812 LEDs successfully changed between red, green, yellow and off.
Buzzer circuit	Active buzzer switched correctly through the S8050 transistor from GPIO 18.
Flame sensor	Analog response changed strongly with flame distance; lower values represented stronger detection in the tested arrangement.
Smoke/gas sensor	Analog readings received through the ADC input; final operating thresholds intentionally left for calibration.
Switch	INPUT_PULLUP logic correctly changed between active and OFF modes.
Combined alarm	LCD, RGB, buzzer and sensor-state logic operated together.
Wi-Fi	ESP32 connected to Wi-Fi and obtained an IP address.

Local API	POST and GET /api/status validated with curl.
Authentication	Missing or incorrect API key returned HTTP 401; valid key returned HTTP 200.
Cloudflare Tunnel	Internet request successfully traversed Cloudflare to Express on the Raspberry Pi.
TLS	ESP32 HTTPS request succeeded with CA validation enabled.
OFF telemetry	Server receives OFF without overwriting the latest sensor readings.
Dashboard	Browser displays latest data and updates automatically.

9. Key Design Decisions

Local-first alarm logic: Detection and outputs remain on the ESP32 rather than relying on the Raspberry Pi. This prevents an Internet failure from disabling the physical alarm.

No persistent database: The project requirement is current-state monitoring rather than historical analytics. Removing a database reduces complexity, setup time and privacy/storage concerns.

Heartbeat + immediate state changes: A heartbeat keeps the server aware that the ESP32 is alive, while changes to WARNING, CRITICAL or OFF are transmitted immediately.

Cloudflare Tunnel instead of port forwarding: The Raspberry Pi is on a separate network from the ESP32. Tunnel routing solves cross-network communication without opening a public inbound port on the home router.

Physical OFF mode: A hardware slide switch gives the user a direct and understandable way to disable sensor evaluation. In OFF mode the ESP32 remains powered so networking and status reporting can continue.

Modular firmware: Separating sensors, outputs, state logic, debug and network code improves readability and makes later changes safer.

10. Limitations and Remaining Work

- Sensor thresholds are prototype values. Final smoke/gas, flame and temperature ranges should be calibrated after the complete hardware has warmed up and stabilized.
- The MQ sensor requires a warm-up period and can vary significantly during startup; production-quality logic should account for stabilization time.
- The current prototype state changes are threshold-based. Additional averaging or consecutive-sample confirmation could reduce false alarms.

- The Raspberry Pi keeps data only in RAM, so restarting the Node.js process removes the latest values. This is intentional for the current scope.
- Automatic startup of the Node.js server has been deferred. cloudflared already runs as a service, but the Express application can later be added as a systemd service.
- Remote OFF control is considered optional future work. If implemented, physical vs remote control priority must be defined clearly before enabling Internet-originated commands.
- This prototype is an educational fire-monitoring system and is not a certified life-safety/fire-alarm product.

11. Conclusion

The project successfully integrates embedded sensing, local alarm control, secure Internet telemetry and a private real-time dashboard. The ESP32 can autonomously evaluate environmental conditions and react locally, while the Raspberry Pi provides remote visibility without becoming a single point of failure for the alarm itself.

The chosen architecture is deliberately lightweight: no database is required, the dashboard presents only the latest state, and Cloudflare Tunnel bridges the two networks without router port forwarding. The remaining work is mainly refinement rather than core functionality: final sensor calibration, optional startup automation, additional false-positive filtering and optional remote OFF control.

Appendix A. Data Model

Example RUNNING payload sent by the ESP32:

```
{
  "system": "RUNNING",
  "state": "NORMAL",
  "temperature": 24.3,
  "humidity": 52.0,
  "smoke": 18,
  "flame": 2150
}
```

Example OFF payload:

```
{
  "system": "OFF",
  "state": "OFF"
}
```

Appendix B. Technology Stack

Layer	Technology
Embedded firmware	C++ using Arduino framework for ESP32
Local device communication	GPIO, ADC, I²C, WS2812 data protocol
ESP32 networking	Wi-Fi, HTTPClient, WiFiClientSecure / TLS

Server	Raspberry Pi + Node.js + Express
Configuration/secrets	dotenv (.env) on Raspberry; secrets.h on ESP32
Public transport	Cloudflare Tunnel + HTTPS
Dashboard	HTML, CSS and JavaScript
Version control	Git / GitHub

End of Delivery Report